DLY0100 DEEP LEARNING · EVALUACIÓN PARCIAL 2

Clasificación de 18 personajes de Los Simpsons con redes convolucionales

98,9%
accuracy en test

95,5%
accuracy en validación

85%
meta de la evaluación

Esteban Gamboa · Felipe Ortega · Pablo Toro

Duoc UC · 2.º semestre 2026

EL PROBLEMA

18 personajes y un desbalance de 6,3 a 1

15.193 · 3.799

imágenes de entrenamiento y de validación (partición 80/20 estratificada)

890

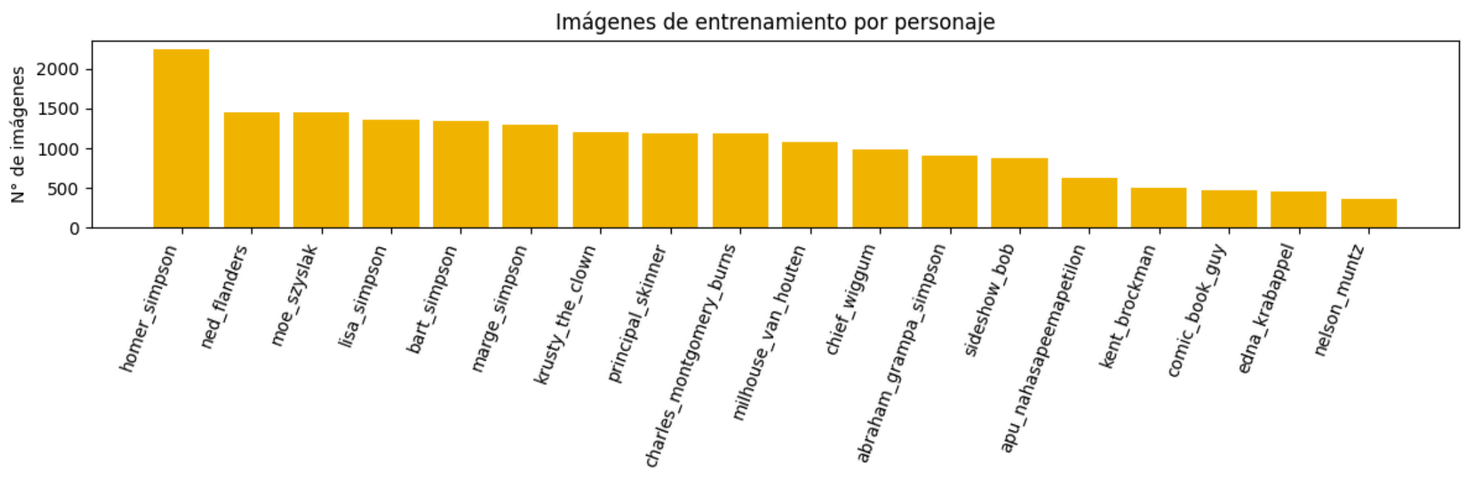
imágenes de test, que no se usan para ninguna decisión

2.246 contra 358

Homer contra Nelson: la clase más grande y la más chica

Meta: 85%

de accuracy en el conjunto de test



Lo compensamos con class_weight: al calcular la pérdida, cada error en Nelson pesa 2,95 y cada error en Homer pesa 0,47.

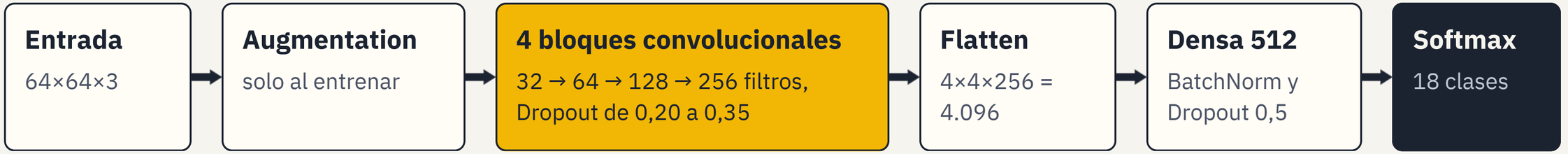
HERRAMIENTAS

Para qué sirvió cada herramienta

Herramienta	Uso	Fortaleza	Limitación
OpenCV	Leer las imágenes y llevarlas a 64×64	Rápida y ya instalada en Colab	A 64×64 se pierden detalles finos de las caras
NumPy	Guardar el dataset y normalizar dividiendo por 255	Opera sobre las 18.992 imágenes de una vez	Todo queda en RAM: cerca de 0,9 GB por copia
scikit-learn	Partición estratificada, class_weight y métricas	Cada tarea en una línea, con funciones probadas	class_weight repondera la pérdida, no crea ejemplos nuevos
Keras 3	Definir, entrenar y evaluar las redes	Augmentation como capas en GPU y callbacks listos	Menos control fino; ImageDataGenerator está obsoleto
Colab	GPU T4 y carga del dataset desde Drive o Kaggle	GPU gratuita y dos fuentes posibles para los datos	Sesión temporal; montar Drive a veces falla

ARQUITECTURA

Una CNN tipo VGG de cuatro bloques



Cada bloque

[Conv 3×3 → BatchNorm → ReLU] dos veces, luego MaxPool 2×2 y Dropout. Dos convoluciones 3×3 seguidas ven lo mismo que una 5×5 con menos parámetros, y duplicar los filtros en cada pooling compensa la resolución que se pierde.

3.283.570 parámetros

64,1% capas densas

35,7% convoluciones

0,2% BatchNorm

Por eso el Dropout más alto, 0,5, va en la capa densa: ahí están la mayoría de los pesos.

CONFIGURACIÓN

Hiperparámetros del modelo final

Hiperparámetro	Valor	Por qué
Optimizador	Adam, learning rate inicial 0,001	Ganó a RMSProp en validación
Batch size	64	Equilibrio entre estabilidad y tiempo
Pérdida	sparse categorical crossentropy	18 clases excluyentes con etiquetas enteras
class_weight	De 0,47 (Homer) a 2,95 (Nelson)	Compensa el desbalance de 6,3 a 1
Augmentation	Volteo horizontal, rotación 0,08, zoom 0,15, traslación 0,1	Que la red nunca vea dos veces la misma imagen
ReduceLROnPlateau	Learning rate a la mitad tras 4 épocas sin mejora	Pasos más finos cerca del mínimo
EarlyStopping	val_loss, paciencia 10, tope de 60 épocas	Restaura la mejor época, no la última

COMPARACIÓN DE MODELOS

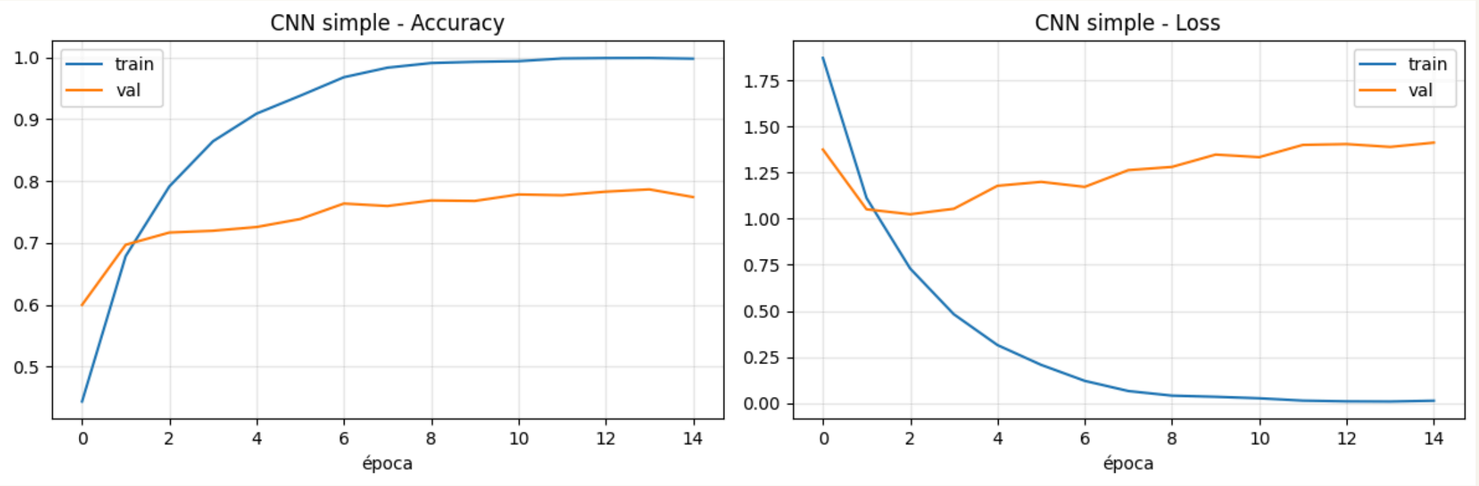
Cinco modelos, las mismas métricas

Modelo	Parámetros	Validación	Test	F1 macro test	Brecha
MLP	6,43 M	19,7%	13,2%	9,2%	2,0
CNN simple (2 conv)	2,12 M	77,4%	93,3%	93,3%	22,4
CNN profunda + BN + Dropout	3,28 M	85,3%	88,3%	88,5%	5,6
Final con Adam (elegido)	3,28 M	95,5%	98,9%	98,9%	3,5
Final con RMSProp	3,28 M	94,8%	97,8%	97,8%	2,7

Decidimos con validación, no con test. El test de este dataset es más fácil que el entrenamiento: todas las CNN aciertan más en test que en validación. La CNN simple llega a 93,3% en test, pero solo a 77,4% en validación.

DIAGNÓSTICO

La CNN simple memoriza



99,9% contra 77,4%

accuracy de entrenamiento contra validación

Época 3

mínimo de la pérdida de validación (1,02); al final sube a 1,41

99%

de los parámetros en una sola capa, la Dense(128)

Le sobra capacidad y le falta regularización. De aquí salen los cambios del modelo siguiente: más pooling para achicar la parte densa, BatchNorm, Dropout y augmentation.

MEJORAS POR AJUSTE

Cada cambio cerró la brecha

PASO 1

MLP

19,7%

validación · brecha 2,0

Capas densas sobre píxeles: no aprende

→

PASO 2

CNN simple

77,4%

validación · brecha 22,4

Convoluciones: aprende, pero memoriza

→

PASO 3

CNN profunda

85,3%

validación · brecha 5,6

Cuatro poolings, BatchNorm, Dropout y class_weight

→

PASO 4

Modelo final

95,5%

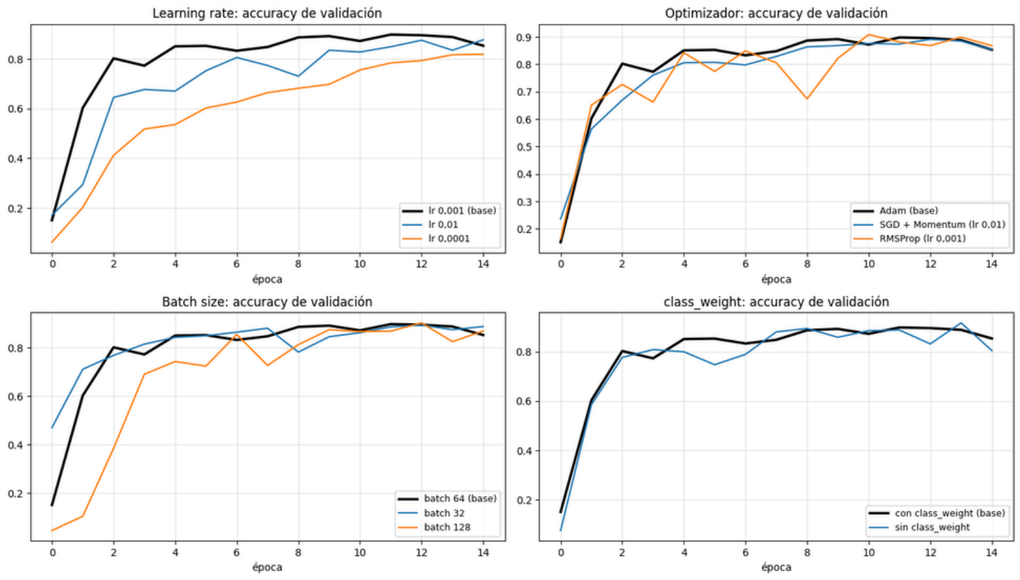
validación · brecha 3,5

Augmentation y reducción del learning rate

+75,8 puntos de validación desde el MLP, y la brecha bajó de 22,4 a 3,5 puntos desde la CNN simple.

AJUSTE DE HIPERPARÁMETROS

Un factor a la vez: cuatro experimentos



CNN profunda sin augmentation, 15 épocas. Comparamos el promedio de validación de las últimas 3 épocas, además de F1 macro y val_loss.

Learning rate

0,001 gana en el promedio y en val_loss; 0,01, en F1. Nos quedamos con 0,001 y lo bajamos durante el entrenamiento.

Optimizador

Empate en el promedio: 87,9%, 87,9% y 87,7%. Como cada uno gana otra métrica, lo decidimos en el modelo final.

Batch size

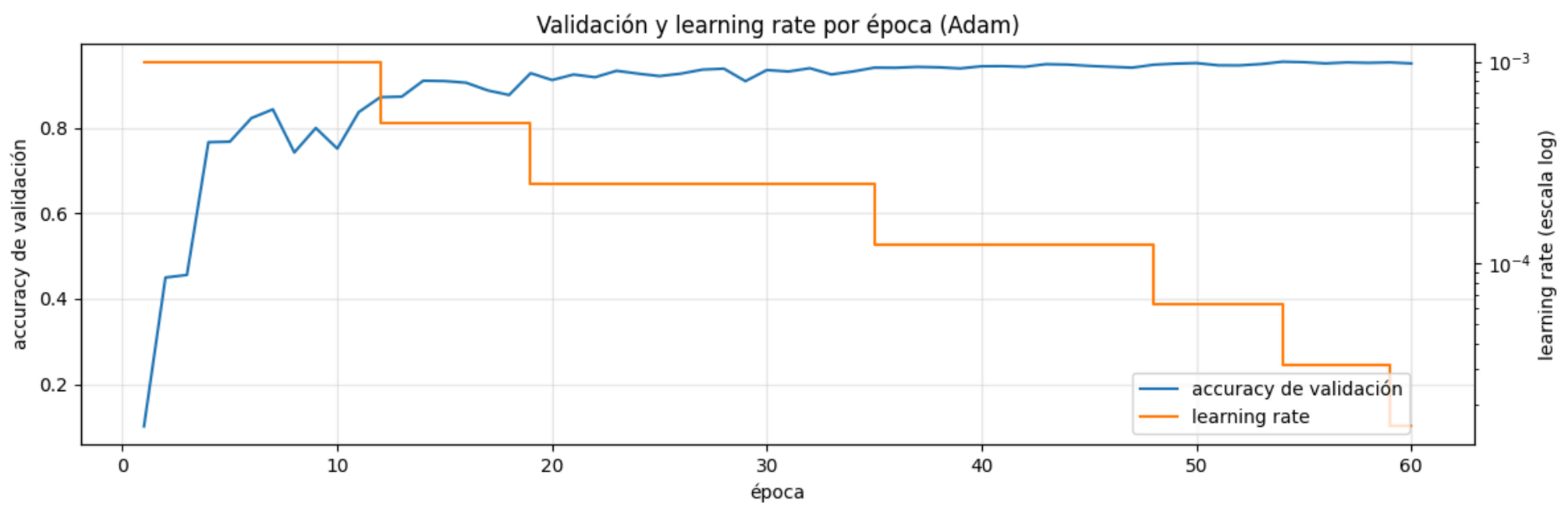
Batch 32 gana en casi todo (88,6% contra 87,9%). Mantuvimos 64; probar 32 queda pendiente.

class_weight: el efecto más claro

Quitarlo baja todas las métricas de validación, y el recall de las clases chicas cae 6 puntos.

ENTRENAMIENTO FINAL

El learning rate bajó seis veces



6 reducciones

en las épocas 12, 19, 35, 48, 54 y 59

+3,5 puntos

con la primera: validación de 83,7% a 87,2%

Época 54

la mejor: val_loss 0,202 y 95,5% de validación

EVALUACIÓN EN TEST

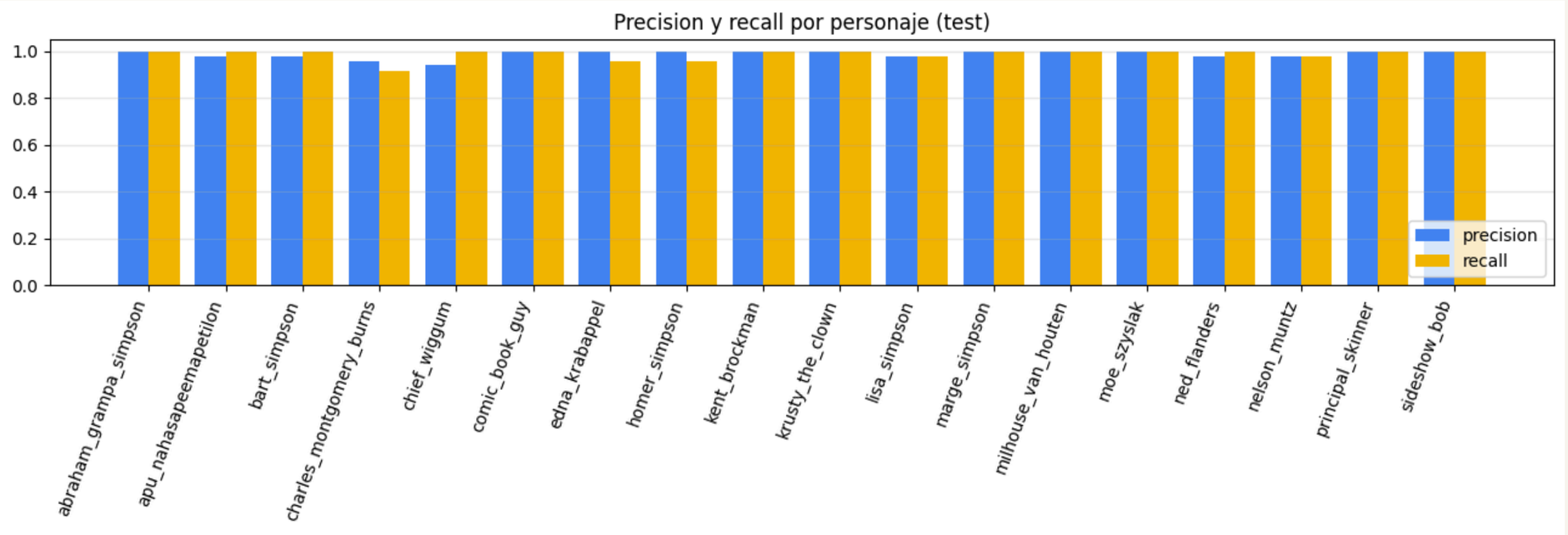
Test: 98,9% de accuracy

98,88%
accuracy en test

98,87%
F1 macro: los 18 personajes por igual

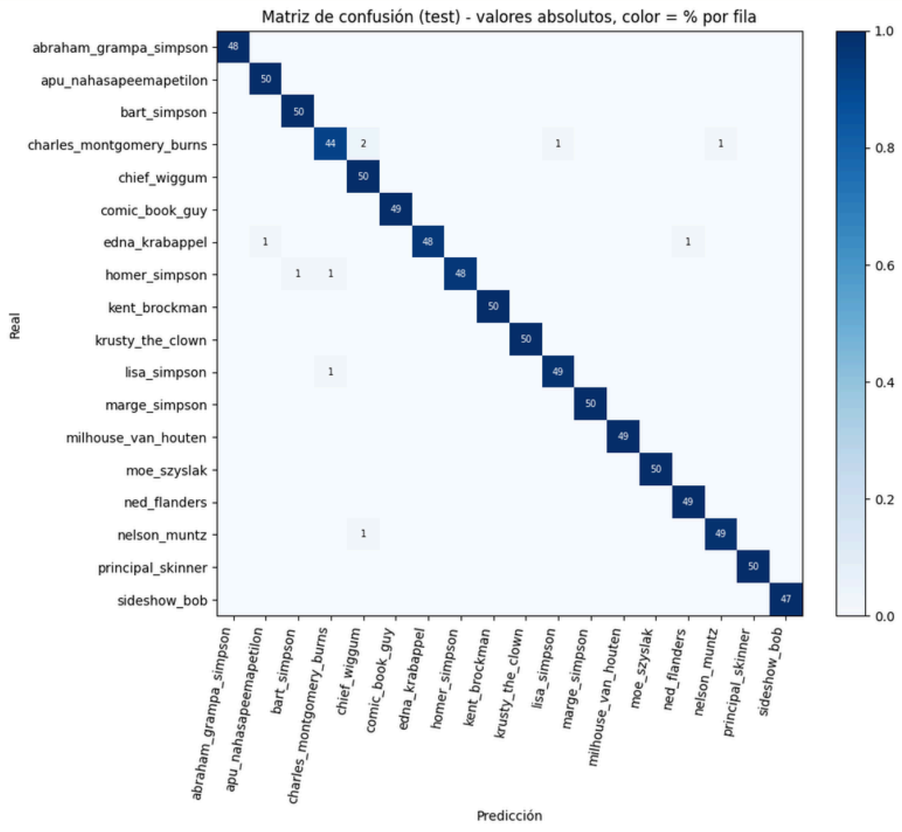
10 de 890
imágenes mal clasificadas

9 de 18
personajes sin ningún error



ANÁLISIS DE ERRORES

Dónde se equivoca el modelo



Burns: 4 de los 10 errores

El peor recall (91,7%). Aparece tapado por una revista, acostado o dentro de un grupo de personas.

Wiggum atrae errores

3 falsos positivos: Nelson con sombrero de copa y frac, y dos imágenes de Burns en grupo.

Cuatro patrones en los errores

- Varios personajes en escena
- Parcial, de espalda o pequeño
- Ropa atípica
- Fondo de color intenso

Con un umbral de confianza de 0,9, el sistema respondería solo el 97,0% de las imágenes, pero con 99,54% de accuracy.

ELECCIÓN DEL MODELO FINAL

Por qué elegimos este modelo

Criterios, en este orden

1. Mejor accuracy de validación
2. Brecha baja entre entrenamiento y validación
3. Mismos parámetros que la CNN profunda
4. El test lo confirma y supera la meta

Por qué no la CNN simple: llega a 93,3% en test, pero tiene 77,4% en validación y 22,4 puntos de brecha. El test la hace parecer mejor de lo que es.

Adam frente a RMSProp

	Adam	RMSProp
Validación	95,53%	94,79%
Test	98,88%	97,75%
Brecha	3,54	2,67
Mejor época	54 de 60	42 de 52

Adam gana por 0,74 puntos de validación. Con augmentation y reducción del learning rate, los dos optimizadores llegan a resultados parecidos.

LIMITACIONES

Limitaciones y próximos pasos

Tres corridas del mismo notebook, tres resultados distintos

class_weight
En una corrida quitarlo mejoraba todo; en esta, lo empeora todo

Adam frente a RMSProp
Diferencia de 2,21, 0,42 y 0,74 puntos de validación

Test
98,54%, 98,20% y 98,88% de accuracy

Lo que limita el resultado

- Una sola semilla por experimento
- SGD con Momentum y batch 32 no se probaron en el modelo final
- El test es fácil y trae 24 imágenes duplicadas
- A 64×64 se pierden detalles de las caras

Lo que haríamos después

- Repetir cada experimento con 3 a 5 semillas
- Transfer learning con MobileNetV2 o EfficientNet
- Mayor resolución: 96×96 o 128×128
- Derivar a revisión manual las predicciones dudosas

CONCLUSIONES

Cuatro ideas para llevarse

1

Las convoluciones son la clave

De 19,7% a 95,5% de validación, con casi la mitad de los parámetros del MLP.

2

El problema era el sobreajuste

Regularizar bajó la brecha de 22,4 a 3,5 puntos.

3

Se decide con validación

El test hacía parecer buena a la CNN simple, que en realidad memorizaba.

4

Una sola corrida no basta

Con una semilla, diferencias de 1 a 3 puntos entre configuraciones no son confiables.

Meta: 85%. Resultado: 95,5% en validación y 98,9% en test.